LOAN DEFAULT PREDICTION SYSTEM

Project Report on the Python Implementation

Mini Project | Supervised Learning / DSML

Abstract

This project builds a supervised machine-learning workflow to estimate whether a bank customer may default on a loan. It uses employment status, bank balance, and annual salary as input features and the Defaulted? column as the target. The code loads and prepares the data, explores class distribution and feature relationships, splits the data into training and testing sets, trains four classification models, compares their performance, and examines feature importance. Because defaults are uncommon, the project considers precision, recall, F1-score, and ROC-AUC in addition to accuracy.

1. Introduction

Loan default prediction can help identify applications that may need additional review. This is a binary classification task: 1 represents a default and 0 represents no default. The system is a decision-support tool and should not be the only basis for approving or rejecting a loan.

2. Problem Statement

The goal is to use available customer information to predict loan default. The supplied project reference describes 10,000 records, including 333 defaults (about 3.33%). A model that predicts no default for every customer could still achieve high accuracy while failing to identify the customers the system is intended to flag.

3. Dataset and Features

The code reads a CSV file using pandas, removes the Index column when present because it is only a row identifier, and renames columns to make them easier to reference in Python. The fields are: Employed (1 if employed, otherwise 0); Bank Balance (current balance); Annual Salary (annual income); and Defaulted? (target: 1 for default, 0 for no default).

4. Data Preparation and Exploration

The program prints the dataset dimensions, checks for missing values, and displays the first few rows. It visualizes the number of default and non-default cases using a count plot. A correlation heatmap helps inspect relationships among numeric columns before model training.

5. Training and Testing

The feature columns are separated from the target. The data is divided into 80% training records and 20% testing records. Stratified sampling approximately preserves the class proportions in both sets, and a fixed random seed makes the split reproducible. StandardScaler is fitted on training data and applied to both sets for scale-sensitive models. Tree-based models use unscaled feature values.

6. Handling Class Imbalance

Logistic Regression, Random Forest, and SVM use class_weight='balanced' so the minority default class receives greater weight during training. XGBoost uses scale_pos_weight, calculated as the number of non-default training cases divided by the number of default training cases. These settings encourage attention to rare defaults but do not guarantee that every default will be detected.

7. Classification Models

Logistic Regression estimates class probabilities using a linear decision function. Random Forest combines predictions from multiple decision trees. SVM with an RBF kernel can model nonlinear decision boundaries; in the updated code, CalibratedClassifierCV wraps SVC to provide probability estimates without relying on the deprecated probability=True parameter. XGBoost is a boosted-tree model that builds trees sequentially.

8. Evaluation Metrics

Accuracy is the proportion of all predictions that are correct. Precision is the proportion of predicted defaults that are actual defaults. Recall is the proportion of actual defaults correctly identified. F1-score is the harmonic mean of precision and recall. ROC-AUC measures how well the model ranks defaults above non-defaults across decision

thresholds. Recall matters when missed defaults are costly, while precision matters because low precision can lead to many false alarms.

9. Visualizations and Model Comparison

The code creates a bar chart comparing accuracy, precision, recall, F1-score, and ROC-AUC. It also plots ROC curves, ranks models by ROC-AUC, and displays a confusion matrix for the model with the highest ROC-AUC. The confusion matrix shows correct and incorrect predictions for each class.

10. Feature Importance

Random Forest and XGBoost provide feature-importance values. The code displays the values as bar charts and prints their ranking. This can help explain which inputs the tree-based models used most, but importance does not prove that a feature causes loan default.

11. Tools and Libraries

The implementation uses Python; pandas and NumPy for data handling; Matplotlib and Seaborn for visualizations; scikit-learn for splitting, scaling, training, and evaluation; and XGBoost for boosted-tree classification.

12. Conclusion

The project demonstrates a full supervised-learning workflow for an imbalanced binary classification problem. It prepares the data, trains four algorithms, compares several metrics, visualizes model performance, and examines feature importance. The code selects the highest ROC-AUC model for closer inspection, but a real lending decision should also consider recall, precision, fairness, and the cost of false alarms versus missed defaults.

Note on Results

The supplied reference report contains results from an earlier configuration. The revised code changes the random seed and some model settings, and the SVM probability handling has been updated. Exact metric values may therefore differ. This report does not present the earlier values as newly measured results. Run the current code and use its actual output for final numerical results.

References

1. Supplied project dataset: Module 3 Project Data.csv. 2. scikit-learn documentation for model selection, preprocessing, classification metrics, SVC, and CalibratedClassifierCV. 3. XGBoost Python package documentation.